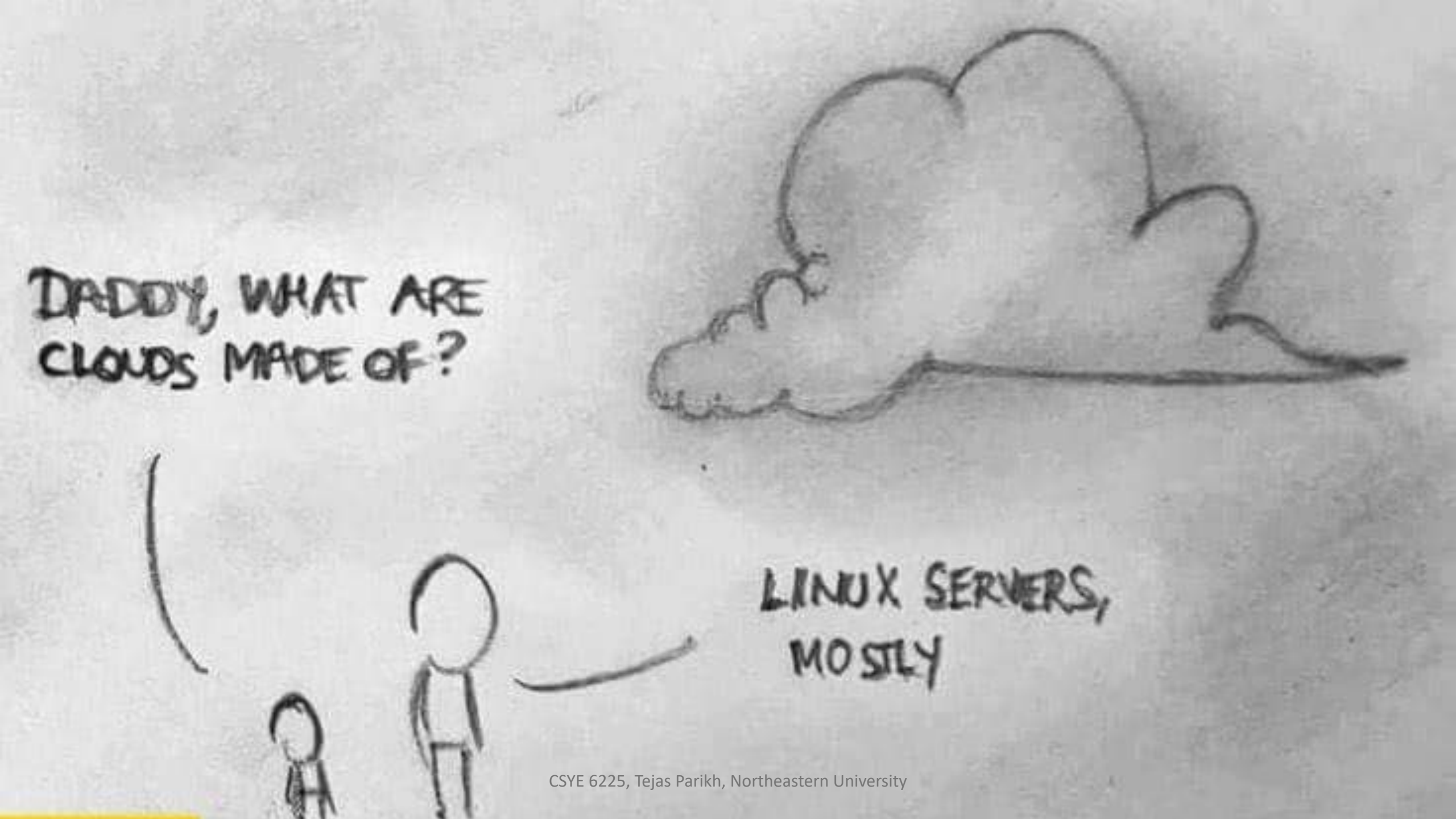
Introduction to Linux Command Line

CSYE 6225 - Boston

Spring 2026

Tejas Parikh (t.parikh@northeastern.edu)


DADDY, WHAT ARE CLOUDS MADE OF?
LINUX SERVERS, MOSTLY

Understanding Linux

What is Linux?

- Linux refers to the operating system **kernel**, not the entire OS
- The kernel is started by the boot loader, which is started by BIOS/UEFI
- Role of the kernel: coordination between hardware and software
- Kernel responsibilities: managing hardware, processes, users, permissions, and file systems
- Runs in "ring zero" (kernel space)

Why Linux Dominates Cloud and Server Environments

- **Open source**: Free to use, modify, and distribute; no licensing costs at scale
- **Stability**: Servers can run for years without rebooting
- **Resource efficiency**: Lightweight; runs well on minimal hardware
- **Security model**: Strong permissions system, rapid security patches
- **Flexibility**: Highly configurable for specific workloads
- Over 90% of public cloud workloads run on Linux

Components of Operating Systems

- **Bootloader**: Manages the boot process
- **Kernel**: Core of the system; manages CPU, memory, and peripherals
- **Daemons**: Background services (web servers, databases, scheduling)
- **Desktop Environment**: User interaction layer *(Note: cloud servers are typically "headless" — no GUI)*
- **Applications**: Software beyond what the desktop environment provides
- **Shell**: Command-line interface for controlling the compute

The Linux Shell

- Acts as an interface between the user and the kernel
- Login program validates credentials, then starts the shell
- Shell is a Command Line Interpreter (CLI)
- Interprets commands and arranges for execution
- Returns prompt after command completion

Popular Linux Shells

- **sh** - Bourne Shell (original)
- **ksh** - Korn Shell
- **csh/tcsh** - C Shell
- **bash** - Bourne-Again Shell *(default on most Linux distributions and cloud instances)*
- **zsh** - Z Shell (modern features, default on macOS)

Note: For this course, assume bash unless told otherwise. When you see shell scripts ending in .sh, they're typically written for bash. If you're on a Mac, your terminal defaults to zsh, but the commands we cover work the same in both.

Bash Special Characters

Character	Meaning
~	Current user's home directory ($HOME)
$	Access a variable (e.g., $HOME)
&	Put a command in the background
*	Wildcard matching zero or more characters
?	Wildcard matching exactly one character
#	Comment (everything after is ignored)
\	Escape character (treat next character literally)

These characters have special meaning to the shell. If you ever need to use them literally (like creating a file with a # in the name), you'll need to escape them with a backslash.

Linux File System Structure

Overview

- Hierarchical structure starting from root (/)
- Everything is a file in Linux (including devices and processes)
  - Everything is a file" is a core Unix philosophy.
  - Your hard drive? It's a file in /dev.
  - Information about running processes? Files in /proc.
  - This uniformity means you can use the same tools (cat, grep, etc.) to inspect almost anything on the system.

Directories You'll Use Often

Directory	Purpose	Notes
`/etc`	System configuration	"Everything to configure" - system-wide config files
`/home`	User home directories	Personal files, application code often deployed here
`/var`	Variable data	Logs (`/var/log`), databases, spool files
`/tmp`	Temporary files	One of few directories regular users can write to
`/opt`	Optional software	Third-party and self-compiled applications
`/usr`	User programs	Applications, libraries, documentation

Other System Directories

Directory	Purpose	Notes
`/bin`	Essential user binaries	Basic commands (often symlink to `/usr/bin`)
`/sbin`	System binaries	Superuser commands (often symlink to `/usr/sbin`)
`/lib`	Shared libraries	Code used by applications (often symlink to `/usr/lib`)
`/boot`	Boot loader files	**DO NOT MODIFY** - system won't start if corrupted
`/dev`	Device files	Generated at boot time or dynamically
`/proc`	Process information (virtual)	CPU info, kernel details - generated dynamically
`/sys`	System information (virtual)	Device information
`/root`	Root user's home	**Separate for security** - avoid modifying
`/run`	Runtime data	Temporary data for system processes
`/srv`	Service data	Web server files, FTP files
`/media`	Removable media mount point	Auto-mounted external storage
`/mnt`	Manual mount point	Legacy; rarely used today

Navigation Commands

pwd - Print Working Directory

- Displays the current directory path
- Concept of "current working directory"
- Understanding directory hierarchy (parent/child)
- Think of pwd as your "You Are Here" marker on a map. When you're lost or confused about where you are in the filesystem, pwd tells you exactly where you're standing.

cd - Change Directory

- Syntax: cd [pathname]
- **Absolute pathnames**: Full path from root (e.g., /home/user/documents)
- **Relative pathnames**: Path relative to current location
- Special notations:
  - . - Current directory
  - .. - Parent directory
  - ~ - Home directory
  - - - Previous directory
- **Absolute paths** start with / and work from anywhere.
- **Relative paths** depend on where you currently are.

ls - List Directory Contents

- Basic usage: ls
- Common options:
  - -l - Long format (permissions, owner, size, date)
  - -a - Show hidden files (files starting with .)
  - -h - Human-readable sizes

Important Filename Facts

- Files beginning with . are hidden
- Filenames are **case-sensitive**
- Linux has no concept of file extensions (they're just conventions)
- Case sensitivity catches Windows users off guard. File.txt, file.txt, and FILE.TXT are three different files on Linux. And unlike Windows, the .txt extension doesn't mean anything to Linux — it's just part of the filename for human convenience.

Productivity Tip: Tab Completion

- Press **Tab** to auto-complete file and directory names
- If multiple matches exist, press **Tab twice** to see all options
- Works for commands, paths, and filenames
- Saves time and reduces typos — use it constantly!

File and Directory Manipulation

cp - Copy Files and Directories

- Syntax: cp [options] source destination
- Common options:
  - -r - Recursive (for directories)
  - -i - Interactive (prompt before overwrite)
  - -v - Verbose
- The **-r** flag is required for directories — without it, cp will refuse to copy a folder. This is a safety feature.
- If you're copying important data, consider using **-i** to get prompted before overwriting existing files.

mv - Move/Rename Files

- Syntax: mv [options] source destination
- Dual purpose: moving AND renaming
- Original file no longer exists after operation
- Unlike cp, the mv command doesn't need -r for directories. Also, there's no separate "rename" command in Linux — renaming is just moving a file to a new name in the same directory.

mkdir - Create Directories

- Syntax: mkdir [options] directory_name
- Option -p creates parent directories as needed.
  - The -p flag is incredibly useful. Instead of creating each directory one at a time, mkdir -p projects/webapp/src/components creates the entire path in one command. It also won't error if the directory already exists.

touch - Create Empty Files or Update Timestamps

- Syntax: touch filename
- Creates an empty file if it doesn't exist
- Updates the modification timestamp if file exists
- Useful for creating placeholder files or triggering file-based workflows
- Despite its simple appearance, touch is useful in scripts. Some build systems watch for file timestamp changes to trigger rebuilds. You can use touch to manually trigger those workflows.

rm - Remove Files and Directories

- Syntax: rm [options] file
- Common options:
  - -r - Recursive (required for directories)
  - -f - Force (no prompts)
  - -i - Interactive

⚠️ **Critical Warnings:**

- **No recycle bin** - deletions are permanent and unrecoverable
- **Never run rm -rf /** - destroys the entire filesystem
- **Be careful with rm -rf *** - deletes everything in current directory
- Always double-check your path before running rm -rf

ln - Create Links

- **Symbolic (soft) links**: Special file containing a pointer to another file
  - Syntax: ln -s target link_name
  - Deleting the link doesn't affect the original
  - Can link across file systems
- **Hard links**: Another name for the same file data
  - Syntax: ln target link_name
  - Indistinguishable from original file
  - Cannot span file systems

ln - When to use each

Link Type	Use Cases
Symbolic	Config file shortcuts, cross-filesystem references, linking to directories
Hard	Backup scenarios, space-efficient file copies (same filesystem only)

find - Search for Files

- Syntax: find [path] [options] [expression]
- Powerful command for locating files based on various criteria

Common usage patterns:

```
find /var/log -name "*.log"              # Find by name pattern
find . -type f -mtime -7                 # Files modified in last 7 days
find . -type d -name "config"            # Find directories named "config"
find /home -size +100M                   # Files larger than 100MB
find . -name "*.tmp" -delete             # Find and delete (use with caution!)
```

I/O Redirection and Pipelines

Standard Streams

- Every process in Linux has three default communication channels.
  - **stdin (0)**: Standard input - keyboard by default
  - **stdout (1)**: Standard output - screen by default
  - **stderr (2)**: Standard error - screen by default
- Understanding these streams is key to understanding redirection.
- When a program prints output, it's writing to **stdout**.
- When it prints an error, it's writing to **stderr**.
- When it reads input, it's reading from **stdin**.
- Redirection lets you change where these streams go.

Output Redirection

- > - Redirect stdout to file (overwrite)
- >> - Redirect stdout to file (append)
- 2> - Redirect stderr to file
- &> or 2>&1 - Redirect both stdout and stderr

Input Redirection

- **<** - Redirect file contents to stdin
- Example: sort < unsorted.txt
- Useful for commands that expect input from keyboard

/dev/null

- The "black hole" of Linux
- Used to discard unwanted output
- Example: command 2>/dev/null

Text Processing Commands

- **cat** - Concatenate and display files
  - Reads files and copies to stdout
  - Can combine multiple files
- **head** - Display first lines
  - Default: first 10 lines
  - Option -n specifies number of lines
- **tail** - Display last lines
  - Default: last 10 lines
  - Option -f follows file in real-time (useful for logs). tail -f is your best friend when troubleshooting. It shows you log entries as they're written in real-time.

grep - Pattern matching

- Syntax: grep [options] pattern file
- Searches for text patterns
- Prints lines containing matches
- Common options:
  - -i - Case insensitive search
  - -r - Recursive search through directories
  - -v - Invert match (show lines that DON'T match)
  - -n - Show line numbers
  - -c - Count matching lines

Pipelines

- Connect commands using | (pipe operator)
- stdout of one command becomes stdin of next
- Example: ls -l | grep ".txt" | head -5

Redirection vs. Pipes

Operator	Purpose	Example
>	Command → File	`ls > files.txt`
|	Command → Command	`ls | grep txt`

Permissions

The Unix Permission Model

- Every file has nine permission bits
- Controls: read, write, execute access
- Three categories: owner, group, others
- Every file has exactly one owner, belongs to exactly one group, and has permissions for "everyone else.”

Permission Bits Structure

```
-rwxr-xr--
│└─┘└─┘└─┘
│  │  │  └── Others: read only
│  │  └───── Group: read and execute
│  └──────── Owner: read, write, execute
└─────────── File type (- = file, d = directory)
```

When you run ls -l, the first column shows permissions. Learn to read this at a glance. The first character tells you the file type (dash for regular file, 'd' for directory, 'l' for symlink). The next nine characters are three groups of three: owner, group, others.

Permission Values

Permission	Symbol	Octal Value
Read	r	4
Write	w	2
Execute	x	1
None	-	0

Permission Values – common combinations

Octal	Symbolic	Typical Use
755	rwxr-xr-x	Executable scripts, directories
644	rw-r--r--	Regular files, config files
600	rw-------	Private files (SSH keys)
777	rwxrwxrwx	⚠️ Avoid

The octal notation comes from adding up the values: read(4) + write(2) + execute(1) = 7 (full access).
So 755 means: owner gets 7 (rwx), group gets 5 (r-x), others get 5 (r-x).
Once you understand the math, you can decode any permission instantly.

Permissions on Directories

Permission	On Files	On Directories
Read (r)	View file contents	List directory contents (`ls`)
Write (w)	Modify file contents	Create/delete files in directory
Execute (x)	Run as program	Enter directory (`cd`)

To access a file, you need execute permission on ALL directories in the path.

chmod - Change File Mode

- Syntax: chmod [mode] file
- Only owner or superuser can change permissions

chmod – octal notation

```
chmod 755 script.sh
chmod 644 config.txt
```

chmod – Symbolic Notation

- Targets
  - u (user/owner)
  - g (group)
  - o (others)
  - a (all)
- Operations
  - + (add)
  - - (remove)
  - = (set exactly)

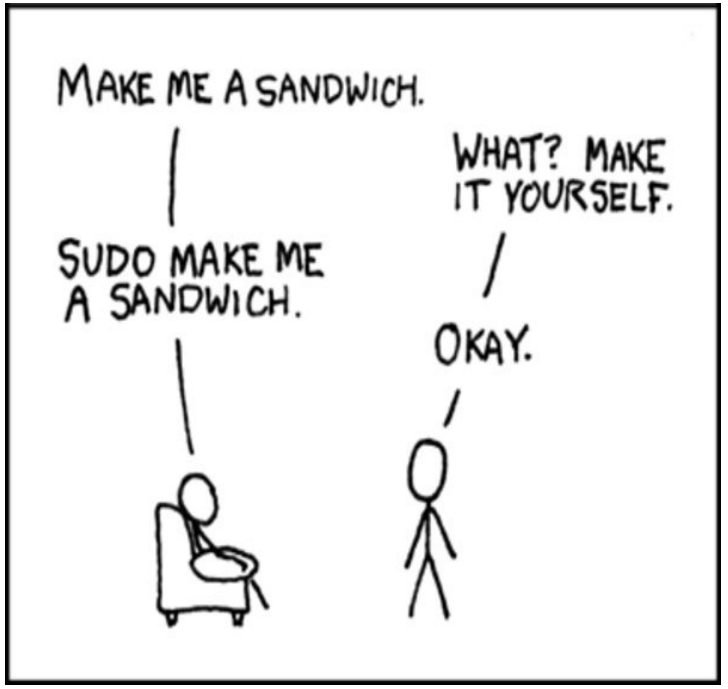
```
chmod u+x script.sh          # Add execute for owner
chmod go-w file.txt          # Remove write from group and others
chmod a+r file.txt           # Add read for everyone
chmod u=rwx,go=rx script.sh  # Same as 755
```

chown - Change Ownership

- Syntax: chown [owner][:group] file
- Requires superuser privileges
- Examples:
  - chown bob file.txt - Change owner
  - chown bob:developers file.txt - Change owner and group
  - chown :developers file.txt - Change group only
- Use -R for recursive (directories): chown -R bob:developers /var/www

sudo - Execute as Superuser

- Allows controlled superuser access
- User authenticates with **their own password**
- Can restrict users to specific commands
- Usage: sudo command
- Sudo permissions defined in **/etc/sudoers**
- **Always edit with visudo** - validates syntax before saving
  - Never edit /etc/sudoers directly with a text editor



Security Warning: Avoid 777 Permissions

- chmod 777 gives everyone full access (read, write, execute)
- Major security risk on servers — anyone can modify or execute files
- Use the most restrictive permissions that still allow your application to function
- If something "needs" 777 to work, the real fix is usually correcting ownership or group membership
- If you find yourself typing chmod 777 to make something work, stop and figure out the real problem. Usually it's that your app is running as the wrong user, or the file ownership is wrong. 777 is the "give up on security" option — never use it in production.

Process Management

ps - View Processes

- Displays snapshot of current processes
- Common usage: ps aux
- Shows: PID, user, CPU%, memory%, command

Understanding ps aux output

```
USER   PID  %CPU %MEM    VSZ   RSS TTY   STAT START   TIME COMMAND
root     1   0.0  0.1 169836 13120 ?     Ss   Jan15   0:05 /sbin/init
```

Column	Meaning
USER	Process owner
PID	Process ID (unique identifier)
%CPU	CPU usage percentage
%MEM	Memory usage percentage
VSZ	Virtual memory size (KB)
RSS	Resident set size - actual RAM used (KB)
TTY	Terminal associated (? = no terminal)
STAT	Process state (S=sleeping, R=running, Z=zombie)
START	Start time
TIME	Total CPU time used
COMMAND	Command that started the process

top - Dynamic Process View

- Real-time, updating view of processes
- Shows system load, memory usage
- Interactive: can sort, kill processes
- Press q to quit, k to kill a process, M to sort by memory

Background Processes

- Append & to run in background
- Example: long_command &
- Adding & lets the command run without blocking your terminal. But if you close the terminal, the background process usually dies too. That's where **nohup** comes in.

nohup - Survive Logout

- Continues running after terminal closes
- Output goes to nohup.out
- Usage: nohup command &

Keyboard Shortcuts for Process Control

Shortcut	Action	Signal
`Ctrl+C`	Interrupt/stop running process	SIGINT (2)
`Ctrl+Z`	Suspend process (pause)	SIGTSTP (20)
`Ctrl+D`	End of input (EOF)	N/A

After Ctrl+Z, use bg or fg to resume the process.

jobs, fg, and bg - Job Control

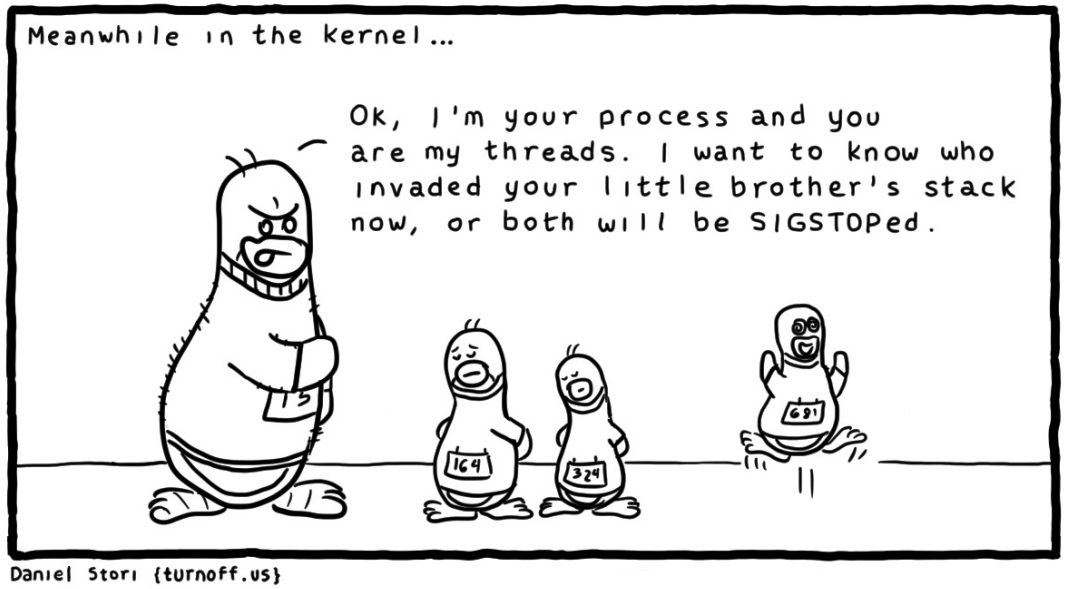
- jobs - List background/suspended processes
- fg [job_id] - Bring process to foreground
- bg [job_id] - Resume suspended process in background

Example workflow:

```
./long_script.sh      # Start a process
# Press Ctrl+Z        # Suspend it
jobs                  # See job list: [1]+ Stopped ./long_script.sh
bg %1                 # Resume job 1 in background
fg %1                 # Bring job 1 back to foreground
```

kill - Terminate Processes

- Syntax: kill [signal] PID
- Use kill -l to list all available signals



Common signals:

Signal	Number	Meaning
SIGHUP	1	Hangup - often used to reload config
SIGINT	2	Interrupt (same as Ctrl+C)
SIGKILL	9	Force kill - cannot be caught or ignored
SIGTERM	15	Graceful termination (default)

Usage examples:

```
kill 1234            # Send SIGTERM to PID 1234
kill -9 1234         # Force kill PID 1234
kill -HUP 1234       # Send SIGHUP (reload config)
killall nginx        # Kill all processes named "nginx"
```

Warning: `kill -9 -1` as root kills ALL processes and crashes the system

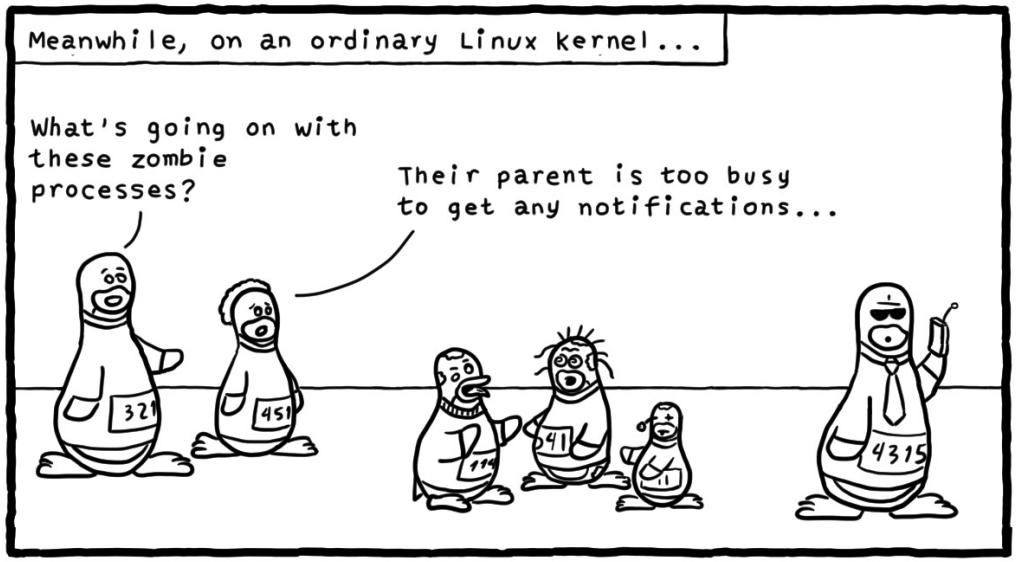
shutdown - System Power Control

- Terminates processes orderly before power off
- Reboot: sudo shutdown -r now
- Power off: sudo shutdown -h now

Zombie Processes

- Process completed but still in process table
- Waiting for parent to read exit status
- Identified with: ps aux | grep 'Z'
- Resolved when parent calls wait()

Zombie processes sound scary but are usually harmless — they're just entries in the process table, not consuming CPU or memory. You can't kill a zombie directly; you have to fix the parent process. A few zombies are normal; thousands indicate a buggy application.



Package Management

What is Package Management?

- Packages are pre-compiled software bundles with dependencies
- Package managers automate installation, updates, and removal
- Repositories: trusted sources of packages maintained by distributions

Common distributions in cloud environments:

Distribution	Package Manager	Where You'll See It
Ubuntu	APT	AWS, GCP, Azure (common default)
Debian	APT	Lightweight server deployments
Amazon Linux	DNF/YUM	AWS EC2 (Amazon's own distro)
RHEL / CentOS / Rocky	DNF/YUM	Enterprise environments

Debian/Ubuntu: APT (Advanced Package Tool)

Updating package lists

```
sudo apt update
```

- Refreshes the list of available packages from repositories
- Should run before installing or upgrading

Upgrading installed packages

```
sudo apt upgrade
```

- Installs newer versions of all installed packages

Installing packages

```
sudo apt install package_name
```

- Downloads and installs package with all dependencies
- Example: `sudo apt install nginx`

Removing Packages (Using APT)

- The difference between **remove** and **purge** matters when you're troubleshooting.
- If you **remove** and reinstall a package and it still doesn't work, maybe the old config files are the problem.
- Use **purge** to completely clean out a package.

Removing packages

```
sudo apt remove package_name      # Keeps config files
sudo apt purge package_name       # Removes config files too
sudo apt autoremove               # Removes unused dependencies
```

Best Practices for Server Package Management

- Always run update before install to get latest versions
- Review what will be installed before confirming
- Keep systems updated for security patches
- Use -y flag cautiously (auto-confirms prompts)
- Use autoremove periodically to clean up unused dependencies

Automated Security Updates

- For production servers, consider enabling automatic security updates:
- **Ubuntu:**
  - sudo apt install unattended-upgrades
  - sudo dpkg-reconfigure unattended-upgrades
- Automatically installs security patches
- Reduces window of vulnerability on servers
- Configure in /etc/apt/apt.conf.d/50unattended-upgrades
- Unattended upgrades are a tradeoff: they keep your system secure automatically, but there's a small risk an update breaks something.
- For most servers, the security benefit outweighs the risk.
- For critical production systems, you might prefer manual updates after testing.

Archives and Compression

tar - Tape Archive

The standard archiving tool on Linux, often combined with compression.

Creating archives

```
tar -cvf archive.tar directory/          # Create archive (no compression)
tar -czvf archive.tar.gz directory/      # Create with gzip compression
tar -cjvf archive.tar.bz2 directory/     # Create with bzip2 compression
```

Extracting archives

```
tar -xvf archive.tar                     # Extract archive
tar -xzvf archive.tar.gz                 # Extract gzip compressed
tar -xjvf archive.tar.bz2                # Extract bzip2 compressed
tar -xzvf archive.tar.gz -C /destination/ # Extract to specific directory
```

Option flags:

Flag	Meaning
`-c`	Create archive
`-x`	Extract archive
`-v`	Verbose (show files being processed)
`-f`	Specify filename (must be last flag before filename)
`-z`	Use gzip compression
`-j`	Use bzip2 compression
`-t`	List contents without extracting

Network Commands

ssh - Secure Shell

- Remote login and command execution over encrypted connection.
- SSH is your gateway to every remote server. When you connect to an EC2 instance, you're using SSH. The connection is encrypted, so even on public networks, your commands and data are secure.

```
ssh user@hostname
ssh user@192.168.1.100
ssh -p 2222 user@hostname                    # Connect on non-standard port
```

scp - Secure Copy

- Copy files between hosts over SSH.

```
# Copy local file to remote
scp file.txt user@host:/path/destination/

# Copy remote file to local
scp user@host:/path/file.txt ./local/

# Copy directory recursively
scp -r local_dir/ user@host:/path/destination/
```

curl and wget - Download Files

Fetch files from URLs.

curl - Transfer data from URLs

```
curl https://example.com                      # Output to terminal
curl -O https://example.com/file.zip          # Save with original filename
curl -o custom.zip https://example.com/f.zip  # Save with custom filename
curl -I https://example.com                   # Fetch headers only
```

wget - Download files

```
wget https://example.com/file.zip             # Download file
wget -O custom.zip https://example.com/f.zip  # Save with custom filename
wget -q https://example.com/file.zip          # Quiet mode
```

Disk Management

df - Disk Free Space

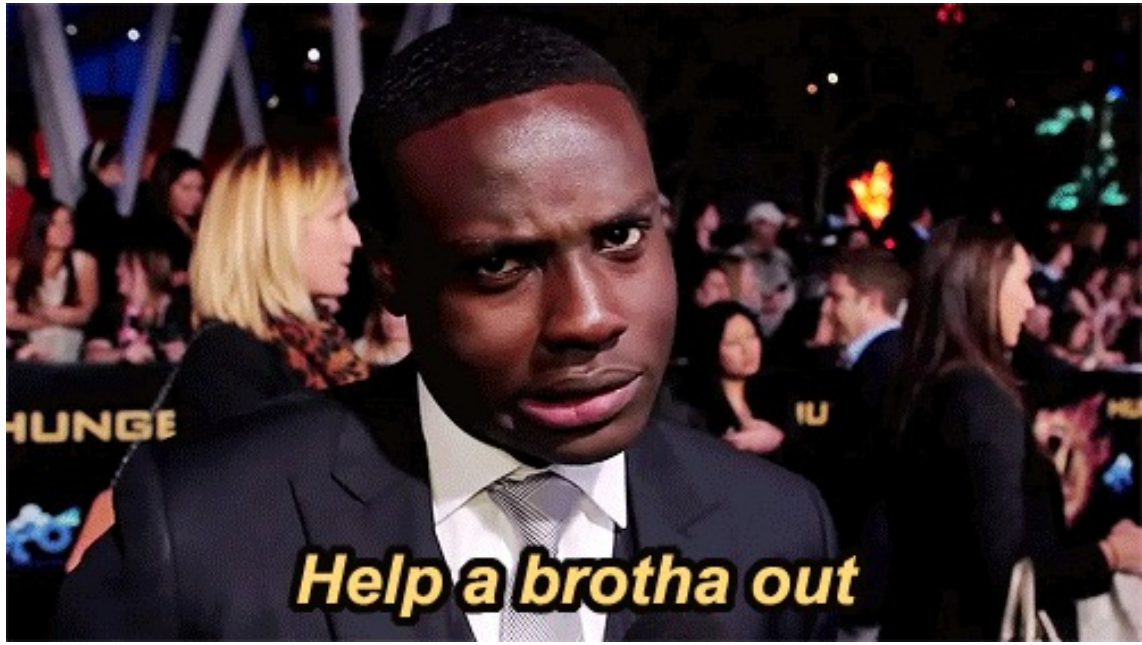
Report file system disk space usage.

```
df -h                                    # Human-readable output (GB, MB)
df -h /                                  # Check specific filesystem
df -h | grep -v tmpfs                    # Exclude temporary filesystems
```

du - Disk Usage

Estimate file and directory space usage.

```
du -sh directory/                    # Summary of directory size
du -sh *                             # Size of each item in current directory
du -sh * | sort -h                   # Sort by size
du -h --max-depth=1                  # Show sizes one level deep
```



Getting Help

man - Manual Pages

- The built-in documentation system.
- Usage: man command_name

Navigation:

Key	Action
`Space` / `Page Down`	Next page
`b` / `Page Up`	Previous page
`/pattern`	Search forward
`n`	Next search result
`q`	Quit

Additional Resources

See course lecture page on Canvas for supplementary materials and documentation links.

LVX
VERITAS
VIRTVS